Hand in QuestPDF

1. The names and student IDs of your group members.
    1. Git Repository `cs452-trains`
    2. Student ID `d273liu`
    3. Student Number 20873700
2. The name of your group's code repository in Gitlab (git.uwaterloo.ca), and the commit SHA for the repository commit that we should review. Your commit must have been created before the assignment deadline.
    1. Git repo link Darcy Liu / cs452-trains · GitLab (uwaterloo.ca)
    2. Sha: `84d0354fa30e772db86dff6c98151d45f920d025`
3. A description of the structure of your kernel so far. We will judge your kernel primarily on the basis of this description. Describe which algorithms and data structures you used and why you chose them.
    - **Send**: The `Send` function allows a process to send a message to a target process, identified by its Task ID (TID). The message is placed in the target process's mailbox, and the sending process may be blocked until a reply is generated.
    - **Receive**: The `Receive` function allows a process to receive a message from its mailbox. If a message is available, the process proceeds with its execution; otherwise, it may be blocked.
    - **Reply**: The `Reply` function is used to reply to a previously received message. It sends a response message back to the original sender. If the receiving process is not expecting a reply, an error is returned.
    - ## Message Queue
      A process's mailbox, used for message storage, is implemented as a circular queue. Messages are placed in this queue, and their processing is managed by the kernel. The maximum capacity of this mailbox is 50 messages.
    - # Data Structure Used:
        - **Circular queue**, due to it's simplicity. I view the queue like a mailbox of sorts. When one receives they take a message from the queue and performs processing with it
    - ## Testing the Send/Receive/Reply
      Run the command `k2pm` from the terminal to test the send/receive and reply time it would out put the average turnaround time in addition to the variance. q
4. A description of your RPS game test - what it does and what the output should show. (We will be running your test.)

Certainly, here's a brief guide on how to use the provided code for Rock, Paper, Scissors (RPS) in your operating system or program:

1. Starting the Game Server:

- To begin, you need to start the game server. You can do this by running the command: `k2rps start`. This command initializes the RPS game environment.

2. Creating RPS Players:

- You can create RPS players by using the `k2rps create` command. The command follows the format: `k2rps create N type`, where:
  - `N` is the number of players you want to create (e.g., 1, 2, 3).
  - `type` specifies the type of player (options: "rock," "paper," "scissors," or "random").

Example: To create three players (one for each type - rock, paper, and scissors), you can run the following commands:

```
k2rps create 1 rock k2rps create 2 paper k2rps create 3 scissors
```

3. Signup for the Game:

- Before playing, players need to sign up to join the RPS game. You can sign up by using the command: `k2rps signup`.

4. Playing the Game:

- Players can play the RPS game by using the `k2rps play` command, followed by their choice (rock, paper, or scissors).
- Example: To play "rock," you can use the command: `k2rps play rock`.

5. Quitting the Game:

- If you want to quit the game, you can use the `k2rps quit` command.

6. Managing the Game:

- To manage the game server, you can use the `k2rps` command with options like "start" (to initialize the game server) and "shutdown" (to shut down the game server).

Example:

- Starting the game server: `k2rps start`
- Shutting down the game server: `k2rps shutdown`

7. Additional Information:

- The code also includes functions for handling messages and tasks in the operating system, which are relevant for the game's implementation. These functions may be used internally by the game.

8. Command Syntax:

- Ensure that you follow the correct command syntax as per the examples provided to create players, play the game, or manage the game server.

Please note that this is a basic guide to get you started with using the code for the Rock, Paper, Scissors game. The code appears to be part of a larger system, so make sure you understand how it integrates with your operating system or program. Detailed documentation and integration instructions may be required depending on your specific use case.
5. A description of your performance measurement methology (how do your tests work?) and a brief summary of conclusions you can draw from your test results.
I measured the time it takes to execute 100 "send" operations and calculated the variance of the individual send times. I observed that, in both the non-optimized and optimized cases, there was a significant amount of variance when using the "receive first" approach. This variance can be attributed to the occurrence of context switches between operations. This phenomenon of context switches affecting performance was evident for both scenarios.

From my test results, I can draw the following conclusions:

1. **Impact of Context Switches**: Context switches between "send" and "receive" operations have a noticeable impact on performance. They introduce variability and overhead in the execution of these operations.
2. **Receive-First vs. Send-First**: The "receive first" approach, in my specific scenario, introduced more variance compared to the "send first" approach. This implies that, in this context, "send first" operations exhibited more predictable and consistent performance.
3. **Compiler Optimization Effect**: The introduction of compiler optimization did not eliminate the variance caused by context switches. This indicates that optimization may not significantly reduce the performance impact of context switches in this particular context.

To address the issues related to variance and context switches, I may explore strategies to reduce context switches, further optimize the code, or implement measures to mitigate the performance impact of context switches in my specific application or environment.